

Lecture 10: Covariance and Correlation, Conditional Expectation and Variance

Jamie Haddock

Table of contents

- 1 Covariance and correlation:: {callout-note icon=false}## Definition: Covariance / correlation

$$\text{Cov}(X, Y) = E[XY] - E[X]E[Y], \quad \rho_{X,Y} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1].$$

Independence $\Rightarrow$ Cov = 0, but not conversely.::: {callout-warning icon=false}##

FactFor independent $X_1, \dots, X_m$: $\text{Var}(\sum_i X_i) = \sum_i \text{Var}(X_i)$ (cross terms vanish).:::—:::

{callout-warning icon=false}## Theorem: Laws of Total Expectation and Variance

$$E[X] = E[E[X | Y]], \quad \text{Var}(X) = E[\text{Var}(X | Y)] + \text{Var}(E[X | Y]).$$

:::## Averaging reduces variance

1

1.1 Reading & HomeworkReading: §4.2-4.4 (Bertsekas & Tsitsiklis, Introduction to Probability). 2

—title: “Lecture 10: Covariance and Correlation, Conditional Expectation and Variance”author: “Jamie Haddock”format: revealjs: output-file: Lecture10_slides slide-number: true preview-links: auto logo: figs/hmc.png css: input/slides.css incremental: true smaller: true code-fold: true embed-resources: true html: code-fold: true embed-resources: true pdf: documentclass: article toc: true number-sections: true geometry: - top=1in - left=1in - bottom=1in - right=1informat-links: falsejupyter: python3filters: - input/remove-pause.luaexecute: echo: true eval: true—

- 1 Covariance and correlation:: {callout-note icon=false}## Definition: Covariance / correlation

$$\text{Cov}(X, Y) = E[XY] - E[X]E[Y], \quad \rho_{X,Y} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1].$$

Independence $\Rightarrow$ Cov = 0, but not conversely.::: {callout-warning icon=false}## FactFor independent $X_1, \dots, X_m$:

$\text{Var}(\sum_i X_i) = \sum_i \text{Var}(X_i)$ (cross terms vanish).:::—::: {callout-warning icon=false}## Theorem: Laws of Total Expectation and Variance

$$E[X] = E[E[X | Y]], \quad \text{Var}(X) = E[\text{Var}(X | Y)] + \text{Var}(E[X | Y]).$$

:::## Averaging reduces variance

```
sigma2 = 4.0ms = [1, 2, 4, 8, 16, 32, 64, 128]empirical_var = [np.var([rng.normal(0, np.sqrt(sigma2), s
```

—# Case Study: Variance Reduction for Trace EstimationAveraging m independent copies of Hutchinson's estimator, $\bar{T} = \frac{1}{m} \sum_k T^{(k)}$, gives $\text{Var}(\bar{T}) = \text{Var}(T)/m$ – today's result, applied to our running example.

```
n = 50A = rng.normal(size=(n, n))def hutchinson_average(m):    total = 0.0    for _ in range(m):
```

1.1 Reading & Homework *Reading: §4.2-4.4 (Bertsekas & Tsitsiklis, Introduction to Probability).*